UNIT-3 Chatgpt [Methods]

Here is a **detailed explanation** of all the mentioned Java **String and StringBuffer methods** with:

- **Syntax**
- **Parameters**
- **Simple and complex examples** (Easy + Complex)
- Grouped by class (String / StringBuffer)

🔵 1. split(regExp) and split(regExp, limit) — String Method

Syntax:

```java
String[] split(String regex)
String[] split(String regex, int limit)
```

Parameters:

- regex: The regular expression (delimiter) to split the string.
- limit: The max number of substrings; the last one contains the rest.

✅ Example 1: Easy

```java
String s = "apple,banana,grape";
String[] fruits = s.split(",");
System.out.println(Arrays.toString(fruits)); // [apple, banana, grape]
```

✅ Example 2: Complex with limit

```java
String s = "one-two-three-four-five";
String[] parts = s.split("-", 3);
System.out.println(Arrays.toString(parts)); // [one, two, three-four-five]
```

🔵 2. regionMatches() — String Method

Syntax:

```java
boolean regionMatches(int toffset, String other, int ooffset, int len)
boolean regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)
```

Parameters:

- toffset: Starting index of first string.
- other: The second string to compare.
- ooffset: Starting index of second string.
- len: Number of characters to compare.
- ignoreCase: Whether to ignore case differences.

✅ Example 1: Easy

```java
String s1 = "HelloWorld";
String s2 = "World";
System.out.println(s1.regionMatches(5, s2, 0, 5)); // true
```

✅ Example 2: Complex with ignoreCase

```java
String s1 = "HelloWORLD";
String s2 = "worldwide";
System.out.println(s1.regionMatches(true, 5, s2, 0, 5)); // true
```

🔵 3. getChars() — String Method

Syntax:

```java
void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)
```

Parameters:

- srcBegin: Start index from the string.
- srcEnd: End index (exclusive).

- dst: Destination char array.
- dstBegin: Starting index in destination array.

✅ Example 1: Easy

```java
String s = "hello";
char[] arr = new char[5];
s.getChars(0, 5, arr, 0);
System.out.println(Arrays.toString(arr)); // [h, e, l, l, o]
```

✅ Example 2: Complex

```java
String s = "programming";
char[] arr = new char[10];
s.getChars(3, 9, arr, 2);
System.out.println(Arrays.toString(arr)); // [ ,  , g, r, a, m, m, i,  ,  ]
```

🔵 4. replace() — String Method

Syntax:

```java
String replace(char oldChar, char newChar)
String replace(CharSequence target, CharSequence replacement)
```

✅ Example 1: Easy

```java
String s = "banana";
System.out.println(s.replace('a', 'o')); // bonono
```

✅ Example 2: Complex with substring replacement

```java
String s = "The sky is blue";
System.out.println(s.replace("blue", "clear")); // The sky is clear
```

🟢 5. insert() — StringBuffer Method

Syntax:

```java
StringBuffer insert(int offset, String str)
```

✅ Example 1: Easy

```java
StringBuffer sb = new StringBuffer("Hello");
sb.insert(5, " World");
System.out.println(sb); // Hello World
```

✅ Example 2: Complex

```java
StringBuffer sb = new StringBuffer("Score:100");
sb.insert(6, " - Grade A");
System.out.println(sb); // Score: - Grade A100
```

🟢 6. deleteCharAt() — StringBuffer Method

Syntax:

```java
StringBuffer deleteCharAt(int index)
```

✅ Example 1: Easy

```java
StringBuffer sb = new StringBuffer("hello");
sb.deleteCharAt(1);
System.out.println(sb); // hllo
```

✅ Example 2: Complex

```java
StringBuffer sb = new StringBuffer("A@B#C$D");
sb.deleteCharAt(1); // remove @
sb.deleteCharAt(2); // remove #
System.out.println(sb); // ABC$D
```

🟢 7. delete() — StringBuffer Method

Syntax:

```java
StringBuffer delete(int start, int end)
```

✅ Example 1: Easy

```java
StringBuffer sb = new StringBuffer("Good Morning");
sb.delete(5, 12);
System.out.println(sb); // Good
```

✅ Example 2: Complex

```java
StringBuffer sb = new StringBuffer("abc123xyz");
sb.delete(3, 6);
System.out.println(sb); // abcxyz
```

🔵 8. substring() — String Method

Syntax:

```java
String substring(int beginIndex)
String substring(int beginIndex, int endIndex)
```

✅ Example 1: Easy

```java
String s = "hello";
System.out.println(s.substring(2)); // llo
```

✅ **Example 2: Complex**

```java
String s = "java programming";
System.out.println(s.substring(5, 16)); // programming
```

🔵 9. lastIndexOf() — String Method

Syntax:

```java
int lastIndexOf(String str)
int lastIndexOf(String str, int fromIndex)
```

✅ **Example 1: Easy**

```java
String s = "hello world hello";
System.out.println(s.lastIndexOf("hello")); // 12
```

✅ **Example 2: Complex**

```java
String s = "abababaab";
System.out.println(s.lastIndexOf("ab", 5)); // 4
```

🔵 10. charAt() — String Method

Syntax:

```java
char charAt(int index)
```

✅ **Example 1: Easy**

```java
String s = "apple";
System.out.println(s.charAt(2)); // p
```

✅ **Example 2: Complex**

```java
String s = "X123Y456Z";
for (int i = 0; i < s.length(); i += 2) {
    System.out.print(s.charAt(i) + " "); // X 2 Y 5 Z
}
```

🔵 11. equals() — String Method

Syntax:

```java
boolean equals(Object anObject)
```

✅ **Example 1: Easy**

```java
String s1 = "java";
String s2 = "java";
System.out.println(s1.equals(s2)); // true
```

✅ **Example 2: Complex**

```java
String s1 = new String("JAVA");
String s2 = "java";
System.out.println(s1.equals(s2)); // false
```

🔵 12. equalsIgnoreCase() — String Method

Syntax:

```java
boolean equalsIgnoreCase(String anotherString)
```

✅ Example 1: Easy

```java
System.out.println("Hello".equalsIgnoreCase("hello")); // true
```

✅ Example 2: Complex

```java
String s1 = "AdminLogin";
String s2 = "adminlogin";
System.out.println(s1.equalsIgnoreCase(s2)); // true
```

🔵 13. compareTo() — String Method

Syntax:

```java
int compareTo(String anotherString)
```

Returns:

- 0 if equal
- <0 if first string < second
- >0 if first string > second

✅ Example 1: Easy

```java
System.out.println("apple".compareTo("banana")); // < 0
```

✅ Example 2: Complex

```java
String s1 = "Zoo";
String s2 = "Ant";
int result = s1.compareTo(s2);
System.out.println(result); // > 0
```

✅ **Summary Table:**

Method	Class	Purpose
split(), split(x,y)	String	Split string using regex
regionMatches()	String	Compare region of strings
getChars()	String	Copy part of string into char array
replace()	String	Replace character/substring
insert()	StringBuffer	Insert at position
deleteCharAt()	StringBuffer	Remove character at index
delete()	StringBuffer	Delete substring
substring()	String	Extract substring
lastIndexOf()	String	Find last occurrence of substring
charAt()	String	Get character at index
equals()	String	Check string equality (case-sensitive)
equalsIgnoreCase()	String	Case-insensitive equality check
compareTo()	String	Compare two strings lexicographically

Here is a **complete explanation** of the following **StringBuffer methods** in Java with:

- ✅ **Syntax**
- ✅ **Parameters**
- ✅ **Simple example**
- ✅ **Complex example**

🟢 indexOf() — *Search first occurrence*

Syntax:

```java
int indexOf(String str)
int indexOf(String str, int fromIndex)
```

✅ Easy:

```java
StringBuffer sb = new StringBuffer("hello world");
System.out.println(sb.indexOf("world")); // 6
```

✅ Complex:

```java
StringBuffer sb = new StringBuffer("one two three two one");
System.out.println(sb.indexOf("two", 5)); // 9
```

🟢 lastIndexOf() — *Search last occurrence*

Syntax:

```java
int lastIndexOf(String str)
int lastIndexOf(String str, int fromIndex)
```

✅ Easy:

```java
StringBuffer sb = new StringBuffer("hello hello");
System.out.println(sb.lastIndexOf("hello")); // 6
```

✅ Complex:

```java
StringBuffer sb = new StringBuffer("abc def abc ghi abc");
System.out.println(sb.lastIndexOf("abc", 15)); // 12
```

🟢 substring() — *Extract part of string*

Syntax:

```java
String substring(int start)
String substring(int start, int end)
```

Note: Although StringBuffer doesn't directly return StringBuffer, it returns a String.

✅ Easy:

```java
StringBuffer sb = new StringBuffer("abcdef");
String sub = sb.substring(2); // from index 2
System.out.println(sub); // cdef
```

✅ Complex:

```java
StringBuffer sb = new StringBuffer("JavaProgramming");
String sub = sb.substring(4, 11);
System.out.println(sub); // Program
```

🟢 charAt() — *Character at specific index*

Syntax:

```java
char charAt(int index)
```

✅ Easy:

```java
StringBuffer sb = new StringBuffer("hello");
System.out.println(sb.charAt(1)); // e
```

✅ **Complex:**

```java
StringBuffer sb = new StringBuffer("ABCDEFG");
for (int i = 0; i < sb.length(); i += 2)
    System.out.print(sb.charAt(i) + " "); // A C E G
```

🟢 **setCharAt() — *Modify character at index***

Syntax:

```java
void setCharAt(int index, char ch)
```

✅ **Easy:**

```java
StringBuffer sb = new StringBuffer("test");
sb.setCharAt(0, 'b');
System.out.println(sb); // best
```

✅ **Complex:**

```java
StringBuffer sb = new StringBuffer("1234567890");
for (int i = 0; i < sb.length(); i += 2)
    sb.setCharAt(i, '*');
System.out.println(sb); // *2*4*6*8*0
```

🟢 **valueOf() — *Convert different types to string***

Syntax:

```java
String valueOf(anyType)
```

This is actually a **String method**, but works well with StringBuffer.

✅ **Easy:**

```java
int num = 100;
String s = String.valueOf(num);
System.out.println(s); // "100"
```

✅ **Complex:**

```java
Object obj = new StringBuffer("bufferText");
String s = String.valueOf(obj);
System.out.println(s); // "bufferText"
```

🟢 **replace() — *Replace a range of characters***

Syntax:

```java
StringBuffer replace(int start, int end, String str)
```

✅ **Easy:**

```java
StringBuffer sb = new StringBuffer("good day");
sb.replace(5, 8, "night");
System.out.println(sb); // good night
```

✅ **Complex:**

```java
StringBuffer sb = new StringBuffer("123456789");
sb.replace(3, 6, "ABC");
System.out.println(sb); // 123ABC789
```

🟢 insert() — *Insert string at position*

Syntax:

```java
StringBuffer insert(int offset, String str)
```

✅ Easy:

```java
StringBuffer sb = new StringBuffer("Hello");
sb.insert(5, " World");
System.out.println(sb); // Hello World
```

✅ Complex:

```java
StringBuffer sb = new StringBuffer("score:50");
sb.insert(6, " out of 100");
System.out.println(sb); // score: out of 10050
```

🟢 reverse() — *Reverse the buffer*

Syntax:

```java
StringBuffer reverse()
```

✅ Easy:

```java
StringBuffer sb = new StringBuffer("abc");
sb.reverse();
System.out.println(sb); // cba
```

✅ **Complex:**

```java
StringBuffer sb = new StringBuffer("1234abcd5678");
System.out.println(sb.reverse()); // 8765dcba4321
```

🟢 delete() — *Delete a range*

Syntax:

```java
StringBuffer delete(int start, int end)
```

✅ **Easy:**

```java
StringBuffer sb = new StringBuffer("abcdef");
sb.delete(1, 4);
System.out.println(sb); // aef
```

✅ **Complex:**

```java
StringBuffer sb = new StringBuffer("remove middle part");
sb.delete(7, 14);
System.out.println(sb); // remove part
```

🟢 deleteCharAt() — *Delete a single character*

Syntax:

java Copy Edit

```java
StringBuffer deleteCharAt(int index)
```

✅ Easy:

java Copy Edit

```java
StringBuffer sb = new StringBuffer("test");
sb.deleteCharAt(2);
System.out.println(sb); // tet
```

✅ Complex:

java Copy Edit

```java
StringBuffer sb = new StringBuffer("1,2,3,4,5");
for (int i = sb.length() - 1; i >= 0; i--) {
    if (sb.charAt(i) == ',') sb.deleteCharAt(i);
}
System.out.println(sb); // 12345
```

📘 Summary Table:

Method	Description	Type
indexOf()	Finds first occurrence	`StringBuffer`
lastIndexOf()	Finds last occurrence	`StringBuffer`
substring()	Returns a portion of string as `String`	`StringBuffer`
charAt()	Returns character at given index	`StringBuffer`
setCharAt()	Sets character at given index	`StringBuffer`
valueOf()	Converts primitives/objects to string	`String`
replace()	Replaces a range of characters	`StringBuffer`
insert()	Inserts text at a given position	`StringBuffer`
reverse()	Reverses characters	`StringBuffer`
delete()	Deletes a range of characters	`StringBuffer`
deleteCharAt()	Deletes a single character	`StringBuffer`

Here is a **concise description** for **all the methods** you mentioned — including those from both String and StringBuffer classes in Java.

✅ **split(String regex)**

Use: Splits the string around matches of the given regular expression.

✅ **split(String regex, int limit)**

Use: Splits the string using a regex, limiting the number of resulting substrings.

✅ **regionMatches(...)**

Use: Compares a region of one string to a region of another, with optional case sensitivity.

✅ **getChars(...)**

Use: Copies characters from the string into a character array.

✅ **replace(char oldChar, char newChar)**

Use: Replaces all occurrences of one character with another.

✅ **equals(Object obj)**

Use: Checks if two strings have the same sequence of characters (case-sensitive).

✅ **equalsIgnoreCase(String anotherString)**

Use: Compares two strings, ignoring case differences.

✅ **compareTo(String anotherString)**

Use: Compares two strings lexicographically (dictionary order).

✅ **indexOf(String str)**

Use: Returns the index of the first occurrence of a substring.

✅ **lastIndexOf(String str)**

Use: Returns the index of the last occurrence of a substring.

✅ **substring(int start) / substring(int start, int end)**

Use: Returns a substring from the string starting from start to end (or end of string).

✅ **charAt(int index)**

Use: Returns the character at the given index.

📘 **StringBuffer Methods**

✅ **setCharAt(int index, char ch)**

Use: Replaces the character at the specified index with the given character.

✅ **valueOf(anyType value)**

Use: Converts a primitive type or object to its string representation.

✅ **replace(int start, int end, String str)**

Use: Replaces characters from start to end with the provided string.

✅ **insert(int offset, String str)**

Use: Inserts the given string at the specified position.

✅ **reverse()**

Use: Reverses the sequence of characters in the buffer.

✅ **delete(int start, int end)**

Use: Removes characters from index start to end.

✅ **deleteCharAt(int index)**

Use: Removes the character at the specified index.

Let me know if you'd like this table in a PDF or a printable format as well!